

CSE722: Applied Cryptography

Lecture – 7: OpenID Connect, OAuth, SSI, WebAuthn/Passkey & Email Security



Inspiring Excellence

Agenda

- OAuth and OpenID Connect
- SSI
- WebAuthn/Passkey
- Email Security

OAuth

- The OAuth (Open Authorization) framework enables a third-party application to obtain limited access to an HTTP service, either
 - on behalf of a resource owner by orchestrating an approval interaction between the resource owner and the HTTP service,
 - or by allowing the third-party application to obtain access on its own behalf
- Its current version is OAuth 2.0 which is defined in RFC6749 and RFC6750

OAuth implementation

- Google: Uses OAuth 2.0 with OpenID Connect support for Gmail, YouTube, Google Drive, Google Calendar, and all Google services
- Facebook/Meta: OAuth 2.0 with OpenID Connect for Facebook Login and Meta services
- Microsoft: OAuth 2.0 with OpenID Connect for Microsoft 365, Azure AD, Outlook, Teams, and Xbox Live
- Twitter/X: Both OAuth 1.0a and 2.0 for Twitter API access
- LinkedIn: OAuth 2.0 with OpenID Connect for professional networking
- Discord: OAuth 2.0 for Discord bots and applications
- GitHub: OAuth 2.0 for repository access and GitHub Apps
- Dropbox: OAuth 1.0 and 2.0 for file storage services
- PayPal: OAuth 2.0 for PayPal API access and payment processing
- Spotify: OAuth 2.0 for music streaming API access

OAuth: four entities/roles

- A Resource Owner is an entity capable of granting access to a protected resource
 - When the resource owner is a person, it is referred to as an end-user
- A Resource Server is the server hosting the protected resources, capable of accepting and responding to protected resource requests using access tokens
- A client is an application making protected resource requests on behalf of the resource owner and with its authorization
 - The term "client" does not imply any particular implementation characteristics (e.g., whether the application executes on a server, a desktop, or other devices)
- An authorization server is the server issuing access tokens to the client after successfully authenticating the resource owner and obtaining authorization

OAuth terminologies

- An authorization grant
 - It is a credential representing the resource owner's authorization (to access its protected resources) used by the client to obtain an access token
 - This specification defines four grant types -- *authorization code*, implicit (a simplified flow), resource owner password credentials (username/password combinations to collect the access token directly) and client credentials (e.g. API key)
- Access token
 - It is a credential used to access protected resources
 - It is a string representing an authorization issued to the client
 - Tokens represent specific scopes and durations of access, granted by the resource owner, and enforced by the resource server and authorization server
- Refresh tokens are issued to the client by the authorization server and are used to obtain a new access token when the current access token becomes invalid or expires

OAuth: abstract protocol flow

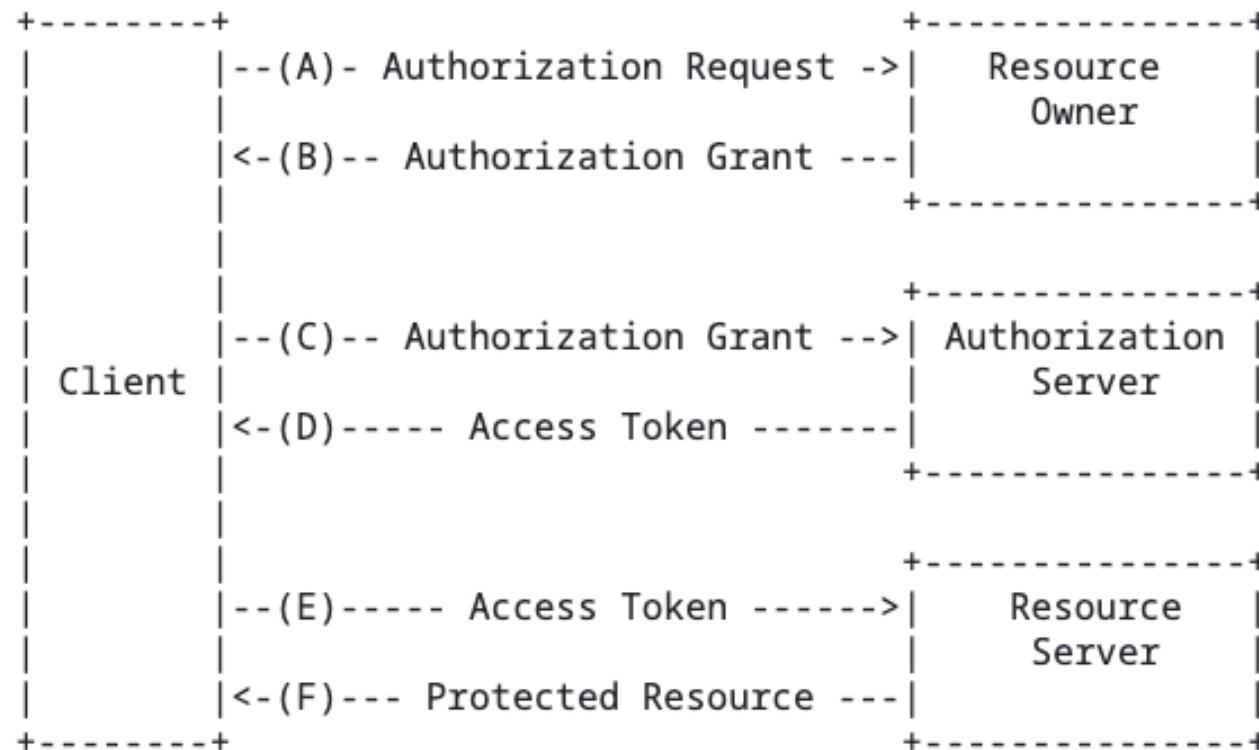


Figure 1: Abstract Protocol Flow

OAuth: abstract protocol flow

1. The *application* requests authorization to access service resources from the *user*
2. If the *user* authorizes the request, the *application* receives an authorization grant
3. The *application* requests an access token from the *authorization server* (API) by presenting authentication of its own identity, and the authorization grant
4. If the application identity is authenticated and the authorization grant is valid, the *authorization server* (API) issues an access token to the application, implying that the authorization is complete
5. The *application* requests the resource from the *resource server* (API) and presents the access token for authentication/authorization
6. If the access token is valid, the *resource server* (API) serves the resource to the *application*

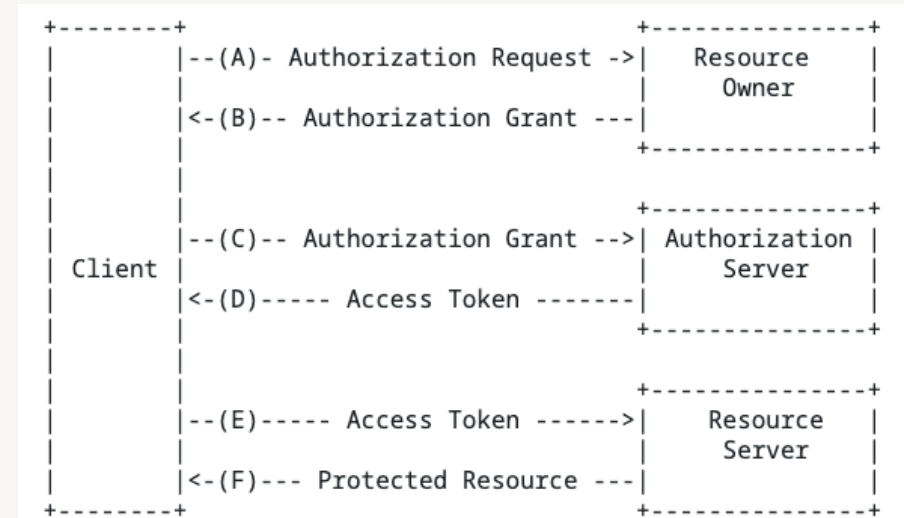
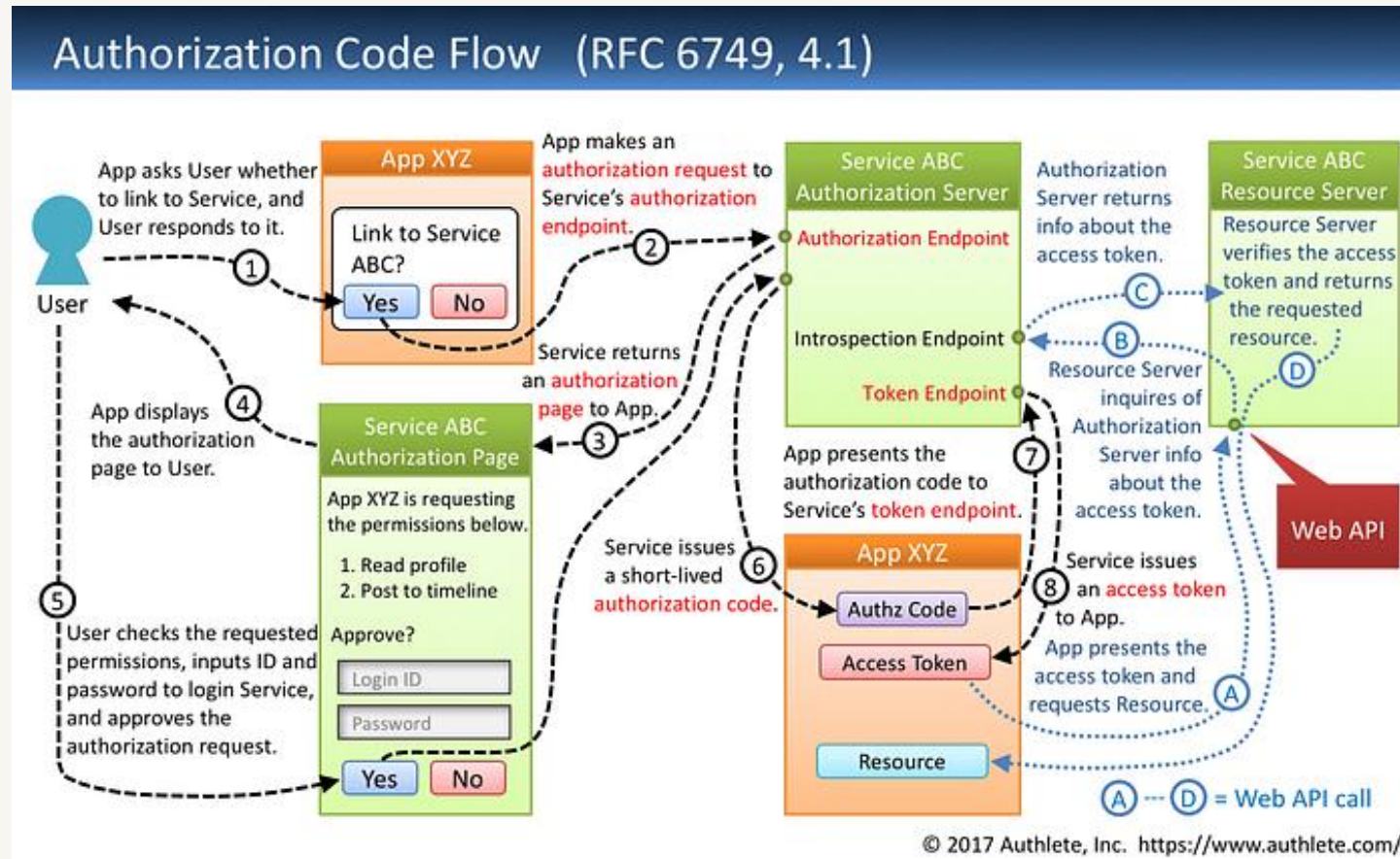


Figure 1: Abstract Protocol Flow

OAuth Flows: Authorization code flow

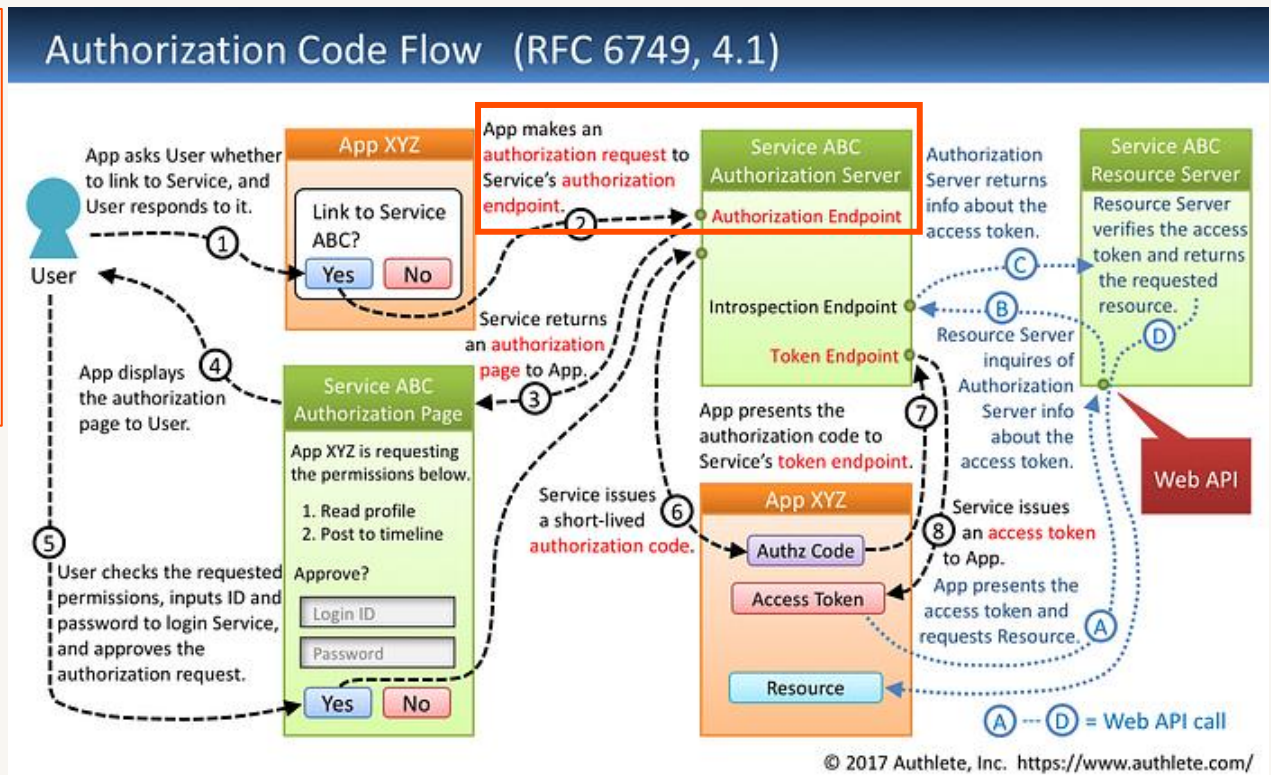


OAuth Flows: Authorization code flow

```
GET {Authorization Endpoint}
?response_type=code      // - Required
&client_id={Client ID}  // - Required
&redirect_uri={Redirect URI} // - Conditionally required
&scope={Scopes}          // - Optional
&state={Arbitrary String} // - Recommended
```

```
HTTP/1.1
HOST: {Authorization Server}
```

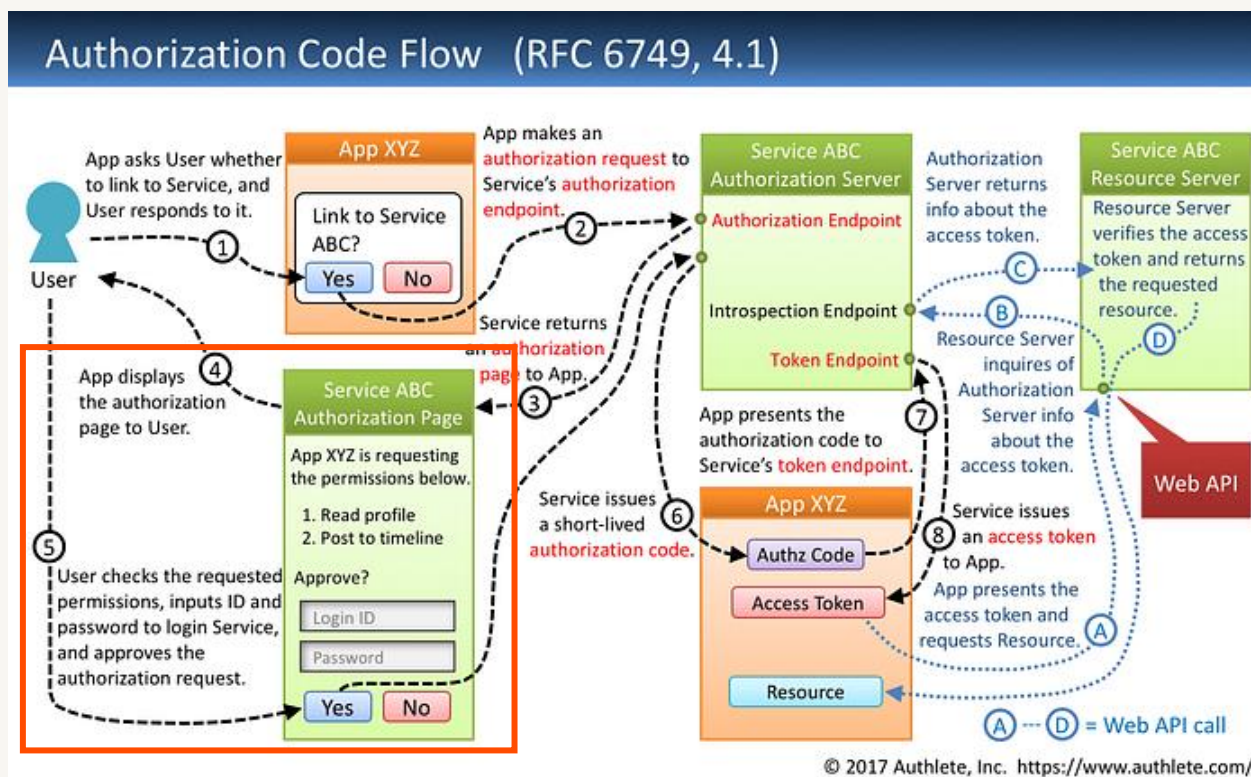
```
GET /authorize?response_type=code&client_id=s6BhdRkqt3&state=xyz
&redirect_uri=https%3A%2F%2Fclient%2Eexample%2Ecom%2Fcb HTTP/1.1
Host: server.example.com
```



OAuth Flows: Authorization code flow

HTTP/1.1 302 Found
Location: {Redirect URI}
?code={Authorization Code} // - Always included
&state={Arbitrary String} // - Included if the authorization
// request included 'state'.

HTTP/1.1 302 Found
Location: https://client.example.com/cb?code=Splxl0BeZQQYbYS6WxSbIA
&state=xyz



OAuth Flows: Authorization code flow

POST {Token Endpoint} HTTP/1.1

Host: {Authorization Server}

Authorization: Basic czZCaGRSa3F0MzpnWDFmQmF0M2JW

Content-Type: application/x-www-form-urlencoded

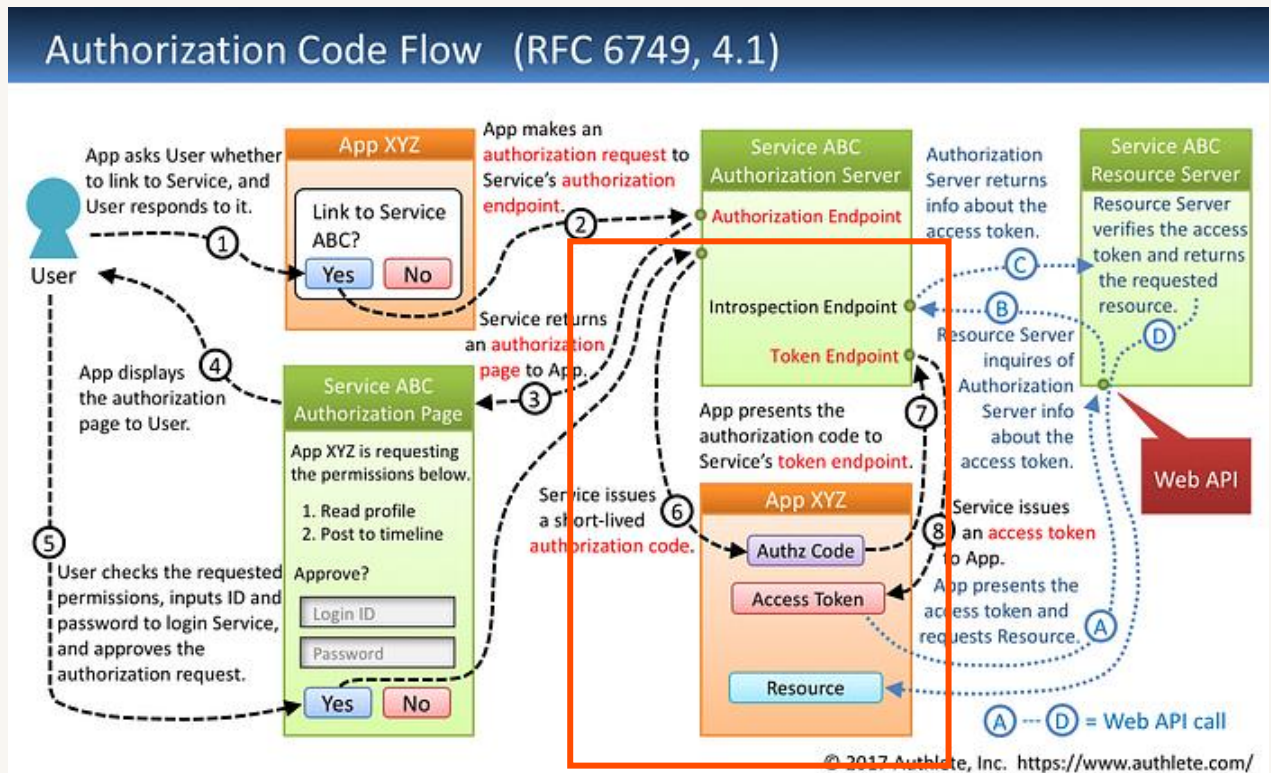
grant_type=authorization_code // - Required
&**code**= {Authorization Code} // - Required
&**redirect_uri**= {Redirect URI} // - Required if the authorization
// request included 'redirect_uri'.

POST /token HTTP/1.1
Host: server.example.com
Authorization: Basic czZCaGRSa3F0MzpnWDFmQmF0M2JW
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code&code=Splxl0BeZQQYbYS6WxSbIA
&redirect_uri=https%3A%2F%2Fclient%2Eexample%2Ecom%2Fcb

Basic indicates a basic username/password authentication
has been used

The code beside basic represents the base64 encoded data
of client id and credential (e.g. API key)



OAuth Flows: Authorization code flow

HTTP/1.1 200 OK

Content-Type: application/json;charset=UTF-8

Cache-Control: no-store

Pragma: no-cache

```
{  
  "access_token": "{Access Token}", // - Always included  
  "token_type": "{Token Type}", // - Always included  
  "expires_in": {Lifetime In Seconds}, // - Optional  
  "refresh_token": "{Refresh Token}", // - Optional  
  "scope": "{Scopes}" // - Mandatory if the granted  
  // scopes differ from the  
  // requested ones.  
}
```

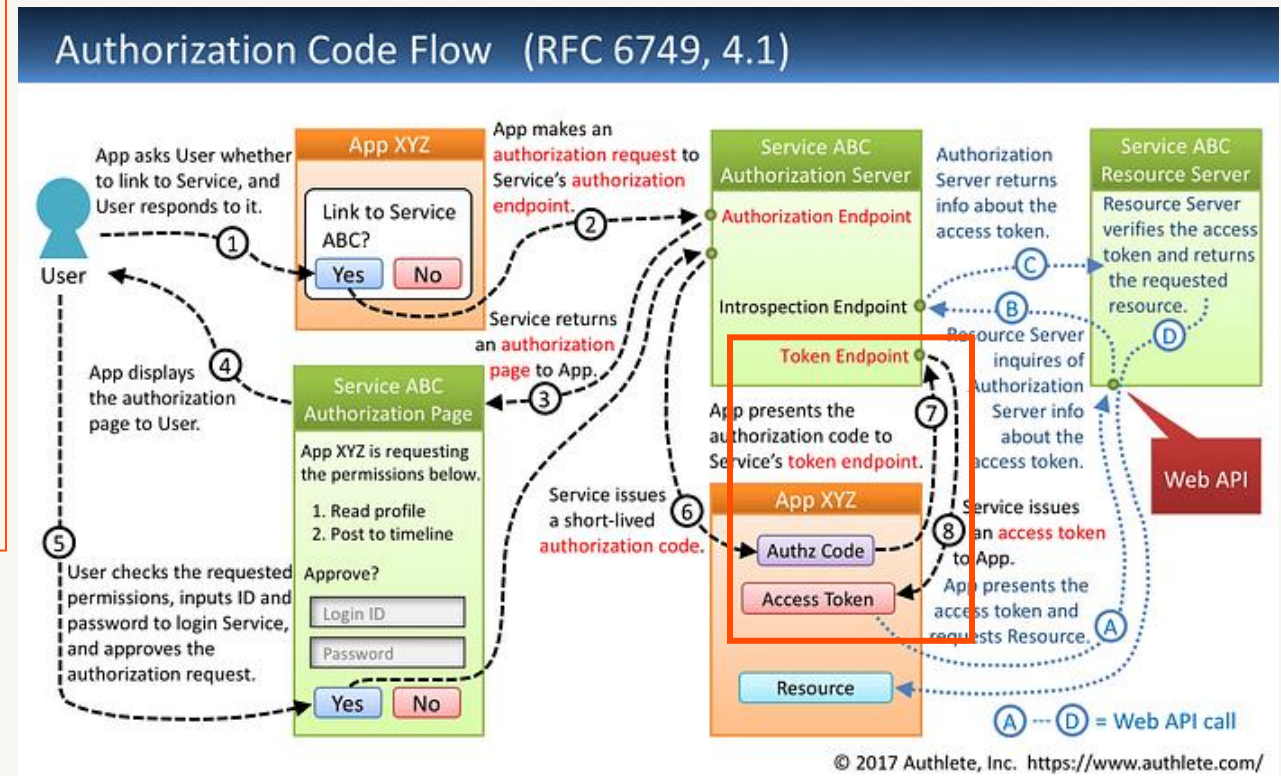
HTTP/1.1 200 OK

Content-Type: application/json;charset=UTF-8

Cache-Control: no-store

Pragma: no-cache

```
{  
  "access_token": "2YotnFZFEjr1zCsicMWpAA",  
  "token_type": "example",  
  "expires_in": 3600,  
  "refresh_token": "tGzv3JOkF0XG5Qx2TlKWIA",  
  "example_parameter": "example_value"  
}
```



OAuth Flows: Authorization code flow

HTTP/1.1 200 OK

Content-Type: application/json;charset=UTF-8

Cache-Control: no-store

Pragma: no-cache

```
{
  "access_token": "{Access Token}", // - Always included
  "token_type": "{Token Type}",      // - Always included
  "expires_in": {Lifetime In Seconds}, // - Optional
  "refresh_token": "{Refresh Token}", // - Optional
  "scope": "{Scopes}"                // - Mandatory if the granted
                                     // scopes differ from the
                                     // requested ones.
}
```

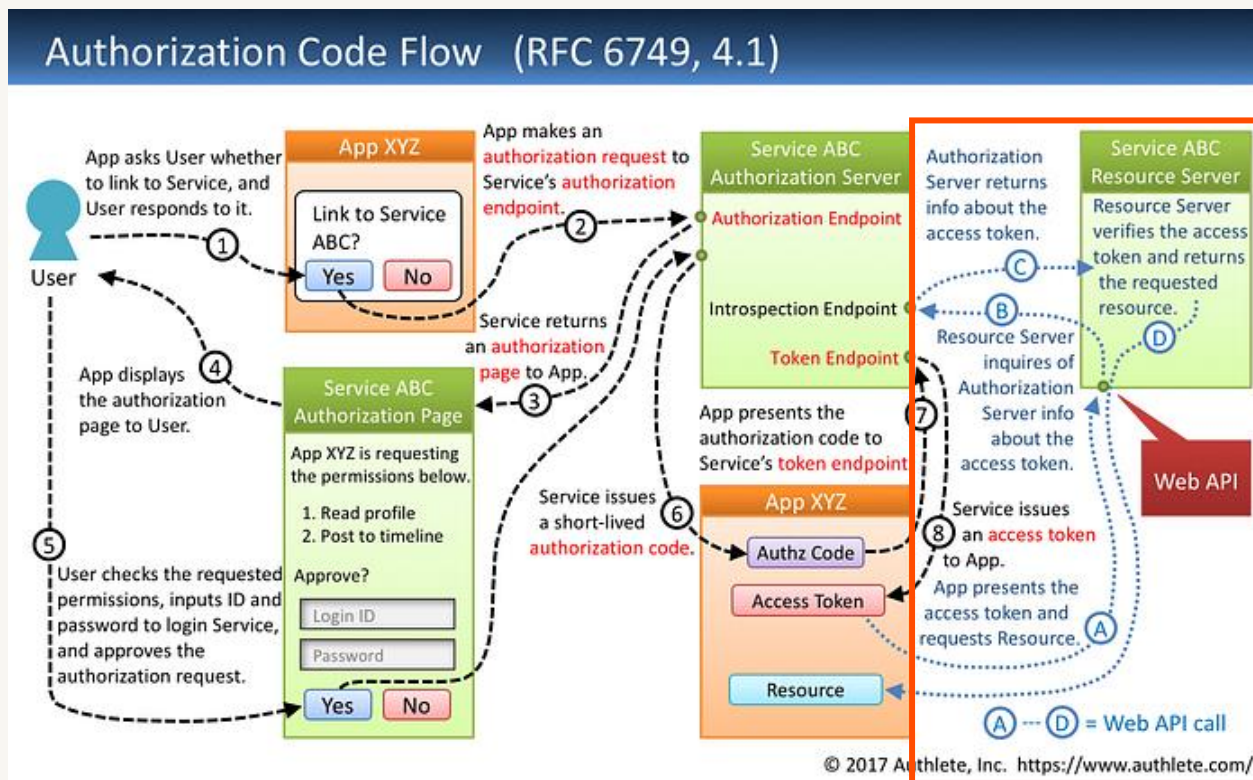
HTTP/1.1 200 OK

Content-Type: application/json;charset=UTF-8

Cache-Control: no-store

Pragma: no-cache

```
{
  "access_token": "2YotnFZFEjr1zCsicMWpAA",
  "token_type": "example",
  "expires_in": 3600,
  "refresh_token": "tGzv3JOkF0XG5Qx2TlKWIA",
  "example_parameter": "example_value"
}
```



OpenID Connect (OIDC)

- OpenID Connect is an interoperable authentication protocol based on the OAuth 2.0 framework of specifications (IETF RFC 6749 and 6750)
- It verifies the identity of users based on the authentication performed by an Authorization Server
- OpenID Connect enables application and website developers to launch sign-in flows and receive verifiable assertions about users across Web-based, mobile, and JavaScript clients
- An assertion is a digitally signed statement (claim) about an end-user

OIDC: entities

- OpenID Provider (OP) or Identity Provider (IdP): an entity that authenticates a user
 - It plays the role of an identity provider (IdP) and can be embedded inside an authorization server
- User: a person that is using a registered client to access resources
- Relying Party (RP): an application or website that outsources its user authentication function to an OP or IdP
- Client: a piece of software that requests tokens either for authenticating a user or for accessing a resource
 - A client must be registered with the OP
 - Clients can be web applications, native mobile and desktop applications, etc.

OIDC: ID token

- An identity token represents the outcome of an authentication process
- It contains at a bare minimum an identifier for the user (called the subject claim) and information about how and when the user was authenticated
- It can contain additional identity data
- It follows a standardized JWT (JSON Web Token) format and contains specific claims defined by the OIDC specification
- It consists of three Base64URL-encoded parts separated by periods:

```
header.payload.signature
```

OIDC: ID token header

- Contains metadata about the token and cryptographic information

```
json
{
  "typ": "JWT",
  "alg": "RS256",
  "kid": "key_identifier"
}
```

- Payload: Contains the claims about the user and authentication event
- Signature: Used to verify the token's integrity and authenticity

OIDC: ID token claim

Core Required Claims 4 5 1 6

Claim	Description	Example
`iss`	Issuer - Identifier for the authorization server that issued the token	`"https://accounts.google.com"`
`sub`	Subject - Unique identifier for the authenticated user	`"auth0"`
`aud`	Audience - Client ID of the application the token is intended for	`"my_client_id"`
`exp`	Expiration Time - Unix timestamp when token expires	`1311281970`
`iat`	Issued At - Unix timestamp when token was issued	`1311280970`

OIDC: entities and ID token

```
GET {Authorization Endpoint}
?response_type=code      // - Required
&client_id={Client ID}  // - Required
&redirect_uri={Redirect URI} // - Conditionally required
&scope=openid {profile|email|address|phone} // - Optional
&state={Arbitrary String} // - Recommended
```

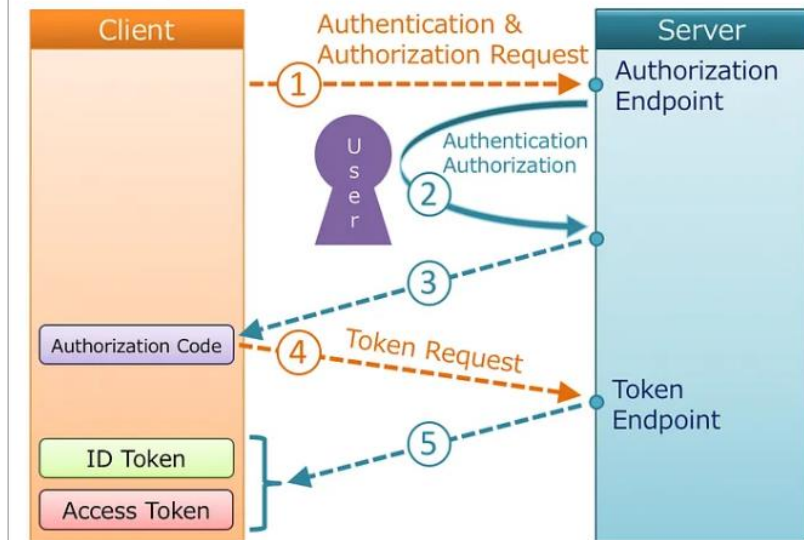
HTTP/1.1
HOST: {Authorization Server}

When `email` scope is requested:

```
json
{
  "email": "jane.doe@example.com",
  "email_verified": true
}
```

Endpoint	Authorization Code	Access Token	ID Token
Authorization	Issued	X	X
Token	X	Issued	Issued

response_type=code (scope includes openid)



OIDC: entities and ID token

```
GET {Authorization Endpoint}
?response_type=code      // - Required
&client_id={Client ID}  // - Required
&redirect_uri={Redirect URI} // - Conditionally required
&scope=openid {profile|email|address|phone} // - Optional
&state={Arbitrary String} // - Recommended
```

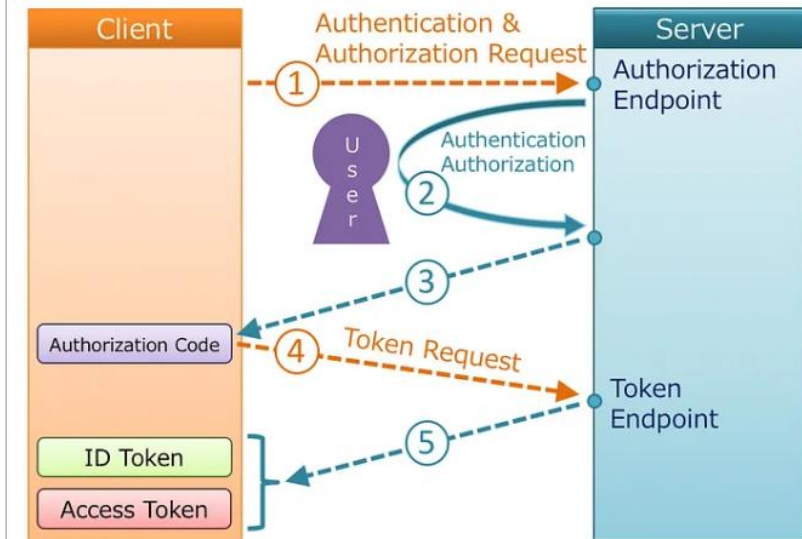
HTTP/1.1

HOST: {Authorization Server}

```
json
{
  "name": "Jane Doe",
  "given_name": "Jane",
  "family_name": "Doe",
  "middle_name": "Marie",
  "nickname": "Janie",
  "picture": "https://example.com/photo.jpg",
  "updated_at": 1311280970,
  "gender": "female",
  "birthdate": "1990-10-31",
  "zoneinfo": "America/New_York",
  "locale": "en-US"
}
```

Endpoint	Authorization Code	Access Token	ID Token
Authorization	Issued	X	X
Token	X	Issued	Issued

response_type=code (scope includes openid)



Decentralised Identity

- Proving the identity of users is a must in many online services
- To create an identity, you need to register to the Service Provider (SP)
- An identity requires a unique identifier to uniquely identify an entity within the system
 - Username -> unique only within a system
 - Email/mobile phone number are universal identifiers
- These identifiers need to be accompanied by a credential (e.g. a password) to prove the ownership of the identifier
- But all these need to rely on a specific SP
 - For emails, it is the Email provider and so on, if such an SP ceases to exist, all services dependent on the identifiers become vulnerable

Decentralised Identity

- Decentralised identity is the solution using public key schemes
 - The public key pk acts as an identity
 - The private key sk is the password to prove the ownership of this identity
- This has some advantages:
 - New identities can be generated at will
 - Also, these new identities cannot be used to uncover your real-world identity, providing a layer of pseudonymous privacy

Self-sovereign Identity (SSI): Motivation

The user is not in control.

Personal data is fragmented across hundreds of silos — each with its own login, privacy policy, and threat surface.

SSI goal:

Give individuals lifetime, portable, cryptographically verifiable control over their own identifiers and credentials



Honeypots

Centralized databases are magnets for breaches.



Surveillance

Identity providers track and profile every login.



Lock-in

Your identity dies when the provider deletes it.



No Portability

Credentials can't move between issuers or verifiers.

Evolution of Identity Management Model

Stage 1	Stage 2	Stage 3	Stage 4
Siloed Each service stores its own user database. <i>Password fatigue. Duplicate data.</i>	Federated An IdP vouches for you across a trust domain. <i>SAML/WS-Fed. IdP sees everything.</i>	User-Centric OAuth/OpenID Connect: delegated access via tokens. <i>Better UX, still IdP-dependent.</i>	SSI User holds credentials; no central IdP. <i>Cryptographic, portable, private.</i>

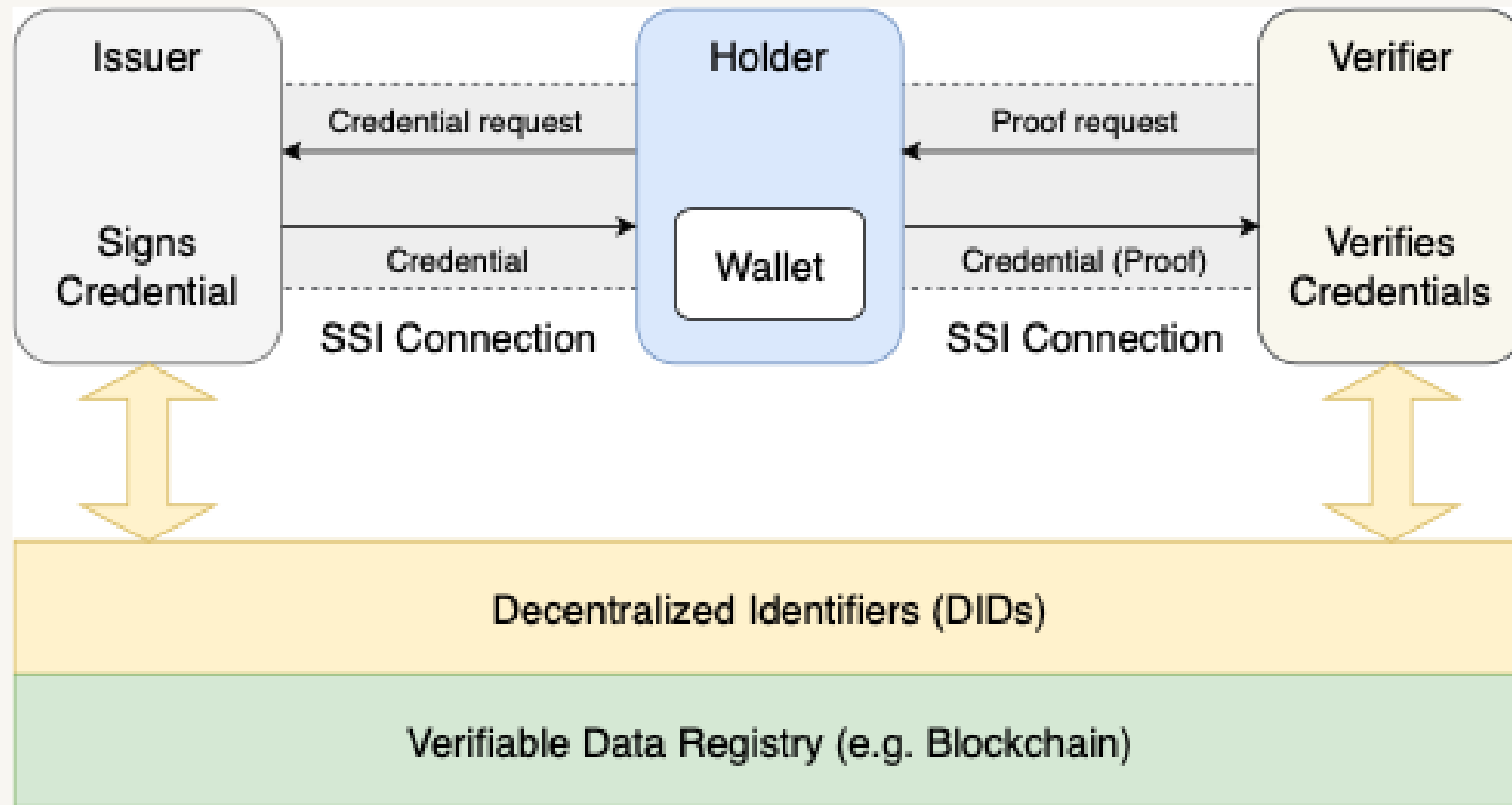
increasing user sovereignty →

SSI vs Federated vs OAuth vs OpenIDC

Dimension	Federated (SAML)	OAuth 2.0	OpenID Connect	SSI
Primary purpose	SSO across domains	Delegated authorization	Authentication over OAuth	Identity without a provider
Identity authority	IdP	Authorization server	OpenID Provider	User + issuer(s)
User sees issuer?	Yes — every login	Yes — authorize dialog	Yes — login prompt	Only at issuance time
Data minimization	Weak (full assertion)	Scoped tokens	Scoped claims	Selective disclosure / ZKP
Offline verification	No	No	No	Yes — signatures verify anywhere
Phone-home on use	Yes	Yes	Yes	No — issuer not contacted
Credential portability	Low	Low	Low	High — user carries them

In OAuth/OIDC, the IdP mediates every login. In SSI, credentials are verified without contacting the issuer.

SSI Architecture



SSI Roles



Issuer

A trusted authority — university, bank, government — that attests to facts about the holder.

Key responsibilities

- Authenticates the holder out-of-band
- Signs a Verifiable Credential
- Publishes its DID + schema to the VDR



Holder

The subject (usually the end user) who receives, stores and presents credentials via a wallet.

Key responsibilities

- Owns private keys in the wallet
- Stores received VCs locally
- Builds Verifiable Presentations



Verifier

A relying party that requests and validates a proof before granting access or service.

Key responsibilities

- Sends a proof request
- Validates signatures & revocation
- Never contacts the issuer directly

SSI Terminologies

- DIDs (Decentralized Identifiers)
- DID Doc (DID Document)
- VCs (Verifiable Credentials)
- VDR (Verifiable Data Registry)
- Wallets and agents
- Mediators

Decetranlized Identifier (DID)

Definition A DID is a URI that identifies a subject without relying on a centralized registration authority. It is generated, owned and controlled by the subject through cryptographic keys.

DID Syntax

did : **sov** : **WRfXPg8dantKVubE3HX8pw**

scheme

Always "did"

DID method (Determines the domain
where this DID is valid)

e.g. sov, key, web, ethr

method-specific ID

Unique string generated from a public key

Cryptographic properties



Key-pair bound

Derived from / anchored to a public key
(Ed25519, secp256k1)



Self-certifying

ID proves control — no registry lookup
required



Verifiable

Holder can sign a nonce to prove
ownership

DID Document (DID Doc)

- A DID Document contains public metadata resolved from a DID
- A JSON/JSON-LD document returned when a DID is resolved
- For every DID created, it is accompanied by a DID Doc which is stored in the respective domain (blockchain, web, database)
- Resolving a DID means retrieving its DID Doc from the respective domain
- It describes how to cryptographically authenticate and interact with the DID subject

Key fields

id The DID itself

verificationMethod Public keys bound to the DID

authentication Keys usable to prove control

assertionMethod Keys usable to sign VCs

keyAgreement Keys for encrypted messaging (DIDComm)

service Endpoints: agents, mediators, hubs

did-document.json

```
{
  "@context": "https://www.w3.org/ns/did/v1",
  "id": "did:sov:WRfXPg8...",
  "verificationMethod": [{
    "id": "did:sov:WRfX...#key-1",
    "type": "Ed25519VerificationKey2018",
    "controller": "did:sov:WRfX...",
    "publicKeyBase58": "H3C2AVvL..."
  }],
  "authentication": ["#key-1"],
  "service": [{
    "type": "DIDCommMessaging",
    "serviceEndpoint": "https://agent.example"
  }]
}
```

VCs

A VC, a W3C Data Model, is a set of temper-evident claims about a subject, created and cryptographically signed by an issuer, so any verifier can check authenticity and integrity — offline.

Structure of a VC

1. Metadata

- Issuer DID
- Credential type / schema
- Issuance & expiry date
- Revocation registry pointer

2. Claims

- Subject DID
- Attributes about the subject
- e.g. name, dateOfBirth, degree
- Encoded as JSON-LD

3. Proof

- Digital signature over metadata + claims
- proofType:
Ed25519Signature2018
- Or BBS+ for selective disclosure
- Links back to issuer's DID Doc key

Wallets and Agents

Wallets are where the holder's keys and credentials actually live

Wallet

- *The holder's personal vault. Usually a mobile app (e.g. Trinsic, Lissi, Aries Bifold)*
- *Issuers and verifiers will also have their own wallets*

Contains

- Private signing & key-agreement keys
- Received VCs (encrypted at rest)
- Connection records (per-relationship DIDs)
- Revocation witnesses

Agent

Software that uses the wallet to speak DIDComm on the holder's behalf. Each party (issuer, holder, verifier) runs its own agent.

Responsibilities

- Encrypt / decrypt DIDComm messages
- Run credential issuance & proof protocols
- Route via mediators (for offline devices)

Verifiable Data Registry (VDR)

The public trust anchor — often a blockchain

A globally readable, tamper-resistant store that anchors the public parts of the ecosystem: DIDs, DID Docs, credential schemas and definitions, and revocation registries. Private data never goes here.

On the VDR

- **DID + DID Doc** — Public keys & service endpoints
- **Schemas** — Attribute names for a credential type
- **Credential defs** — Issuer's keys for a schema
- **Revocation registry** — Status accumulator or list

NOT on the VDR

VCs themselves, PII, holder DIDs used in pairwise relationships.

Implementations

Hyperledger Indy Purpose-built permissioned ledger for SSI. Used by Sovrin.

Ethereum (did:ethr) Smart-contract-based DID method.

Bitcoin (did:btcr) Anchors DIDs in Bitcoin transactions.

did:web DNS + HTTPS server — no blockchain at all.

IPFS / Sidetree Content-addressed, layer-2 DID methods.

SSI Reference Implementation Hyperledger SSI Stack

Aries

Agents, protocols, wallet API

Ursa

Shared cryptographic library

Indy

Distributed ledger + DID method

Hyperledger SSI Stack

What Aries provides

Agent frameworks

ACA-Py (Python), Aries Framework JavaScript (Credo), .NET, Go.

RFC protocols

Aries Interop Profile defines versioned wire protocols.

Wallet interface

Pluggable secure-storage (Indy SDK, Askar).

DIDComm messaging

Transport-agnostic secure channel between agents.

Proof & issuance

Implements issue-credential and present-proof protocols.

DIDComm Protocol

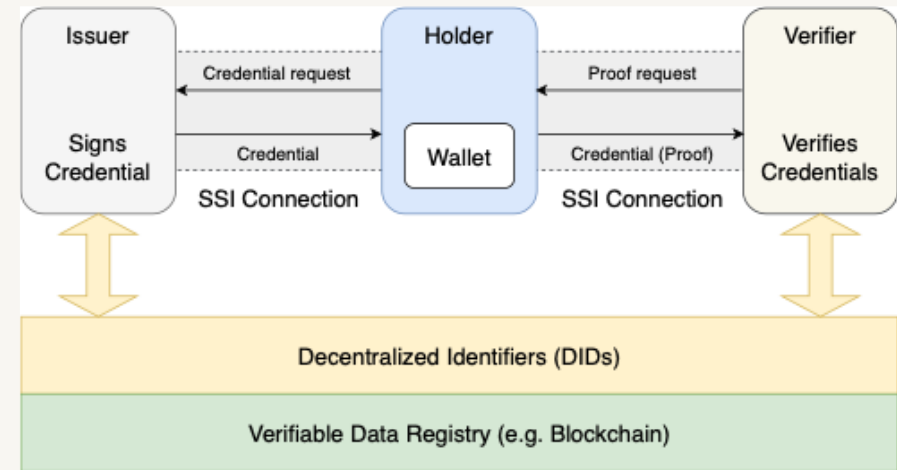
- DIDComm (Decentralized Identifier Communication) is a secure messaging protocol designed for peer-to-peer, decentralized communication using Decentralized Identifiers (DIDs)
- It is widely used in self-sovereign identity (SSI) systems
- It enables private, authenticated, and encrypted messaging and is built on top of:
 - Decentralized Identifiers
 - Public Key Cryptography
- Developed under the Decentralized Identity Foundation (<https://identity.foundation/>)
- Its primary goal is to enable secure communication without relying on centralized infrastructure
- At its core, DIDComm creates a secure "envelope" for messages ensuring that even if a message travels over insecure transport channels (like a public Wi-Fi or a standard HTTP request), the contents remain confidential (encrypted channel), authenticated, and tamper-proof

DIDComm Protocol Flow

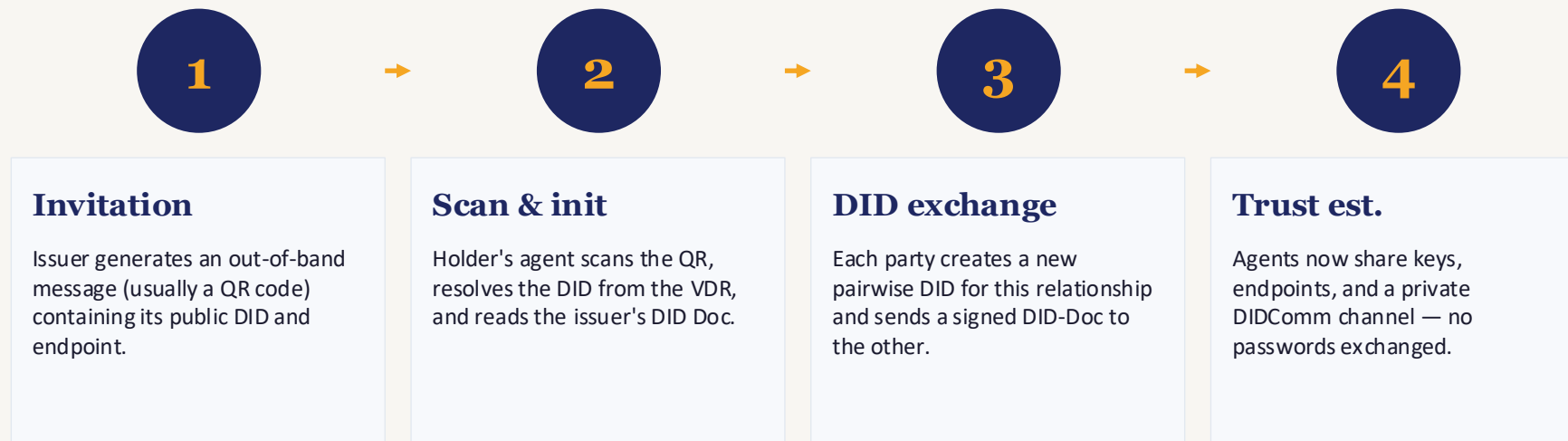
- Resolution: The sender uses the recipient's DID to look up their "DID Document" (usually on a distributed ledger)
- Key Exchange: The sender finds the recipient's public key (Key Agreement) in that DID Document
- Construction: The sender constructs the message (e.g. its DID), encrypts it for the recipient's key, signs it with their own private key, and puts it in the DIDComm envelope
- Transport: The sender transmits the message via the transport layer (e.g., HTTP POST)
- Unpacking: The recipient uses their private key to decrypt and verify the signature, ensuring the message came from the expected party and wasn't altered in transit

SSI Steps

- Establishing an SSI connection
- Providing a VC
- VC storage
- Requesting and presenting a proof



Step 1: Establishing a connection



Cryptographic outcome

Both parties hold a pairwise DID and know the other's public key. This forms a private DIDComm channel — unique to this relationship and unlinkable to any other.

Step 2 – Step 4

- Providing a VC: Once the connection is established, on request from the user (holder), the issuer issues a VC which is transferred via the previously established SSI connection
- Step 2 VC storage: The user verifies the validity of the VC by verifying the digital signature in the VC. After a successful verification, the VC is stored in an SSI wallet of the user
- Step 3 Requesting proof: When the user tries to access a service provided by the verifier (assuming the verifier is acting as an SP), the verifier requests a proof to the user
 - The user then releases the VC using the established SSI connection between the user and the verifier
 - The verifier verifies the digital signature in the presented VC and then takes an authorisation decision to allow/deny a user

Mediators

- A mediator is a cloud-based agent that acts as a persistent, always-online relay for another agent (usually an edge agent) that cannot reliably receive messages on its own
- It is a purely optional infrastructure component, but in practice almost every mobile SSI wallet depends on one
- Mobile wallets often have the following issues:
 - Availability: Phones go offline (Airplane mode, battery saver, etc.).
 - Reachability: Mobile devices frequently change IP addresses, making it difficult for other agents to "find" them on the internet.
 - Connectivity: Corporate firewalls or carrier-grade NATs often block unsolicited incoming connections.

Mediator workflows

- Provides a public endpoint
 - The mediator runs on a server with a stable URL (e.g., <https://mediator.example.com>)
 - That URL is what gets published in the holder's DID Document as the `serviceEndpoint` – so when anyone resolves the holder's DID, they are pointed at the mediator, not at the phone
- Receives encrypted messages on the holder's behalf
 - An issuer or verifier sends a DIDComm message addressed to the holder's DID
 - The message is double-wrapped: the inner envelope is encrypted to the holder's key, and the outer envelope is a forward message encrypted to the mediator
 - The mediator peels off the outer layer but cannot read the inner payload – it only sees a routing instruction and an opaque ciphertext

Mediator workflows

- Queues the message
 - Because the holder is usually offline, the mediator stores the ciphertext until the holder reconnects
- Delivers to the edge agent via one of several mechanisms:
 - Pickup protocol – the wallet polls or long-polls the mediator when it comes online
 - Push notification – the mediator triggers the wallet to fetch data periodically

SSI key takeaways



User holds the keys

SSI inverts the IdP model — the holder's wallet is the root of identity, not a central database.



Trust through cryptography

VCs are verified by signatures over issuer DIDs resolved from the VDR — no runtime call to the issuer.



Selective disclosure is native

ZKP-based proofs let the holder reveal only what a verifier needs, preserving privacy by design.



Standards and open stacks

W3C DIDs & VCs, DIDComm, Hyperledger Aries/Indy/Ursa form an interoperable, implementable foundation.

Authentication using passwords

- The common flow when using password-based authentication
 - i. Create a password
 - ii. The password is hashed and salted and then sent up to the server
 - iii. If the same value is ever produced in the future, you're authenticated into the system

Authentication using passwords

- Ultimately, the server is responsible for storing this password /storage derivation – This makes the server a target for attackers
- In addition to the server being a target, other issues include:
 - Password reuse
 - Weak passwords
 - Credential leaks
 - Phishing

Passwords

81% of breaches leveraged either stolen and/or weak passwords.



Source: **2017 Verizon Data Breach Investigations Report**

Passwords

"The average... user has over 107 accounts registered to one email address... In 2020, the average number of accounts per internet user will be 207"

- Dashlane, 2015

Guess the best solution is to buy a password manager, right?

Definitely not get rid of passwords...

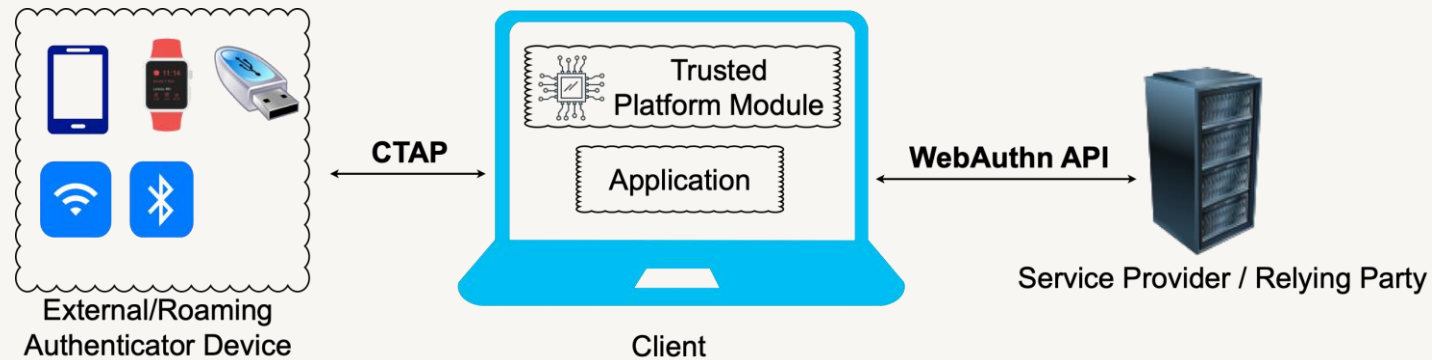
WebAuthn

- A W3C standard enabling passwordless authentication
- Replaces weak secrets (passwords) with strong cryptographic proofs
- Part of the FIDO2 ecosystem (WebAuthn + CTAP)
- Key Philosophy: User-centric security that eliminates phishing and the use of passwords

WebAuthn: entities

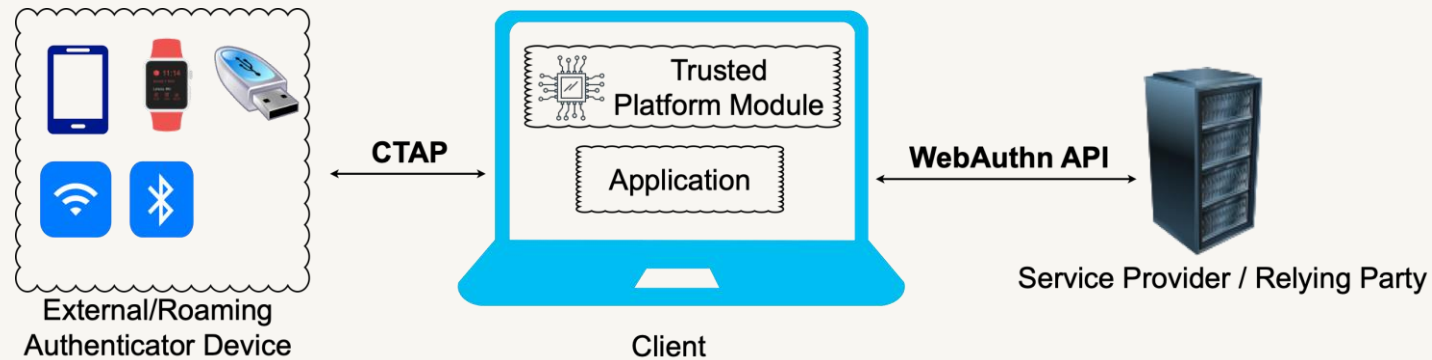
- Relying Party (RP): The server/website requesting authentication
- Client/Browser: Facilitates communication between RP and Authenticator
- Authenticator: Hardware/Software (TPM, Secure Enclave, YubiKey) that stores private keys, acting as the 'wallets'

WebAuthn: core architecture



- First, users try to register an account with the relying party and the relying party sends the user a challenge which is relayed to the authenticator using the Client to Authenticator Protocol (CTAP2)
- The authenticator then generates a new key pair specific to the relying party and using it, the challenge is signed
- Then, it returns the signature, public key, and some attestation information to the relying party and securely stores the keys against the account

WebAuthn: core architecture



- After registering, when the user wants to access the services from that relying party, the relying party sends a challenge to the client which is again passed to the authenticator
- The authenticator signs the challenge using previously created keys and returns it to the relying party
- The user is authenticated once the signature is verified using the stored public key, and access may then be granted based on authorization rules

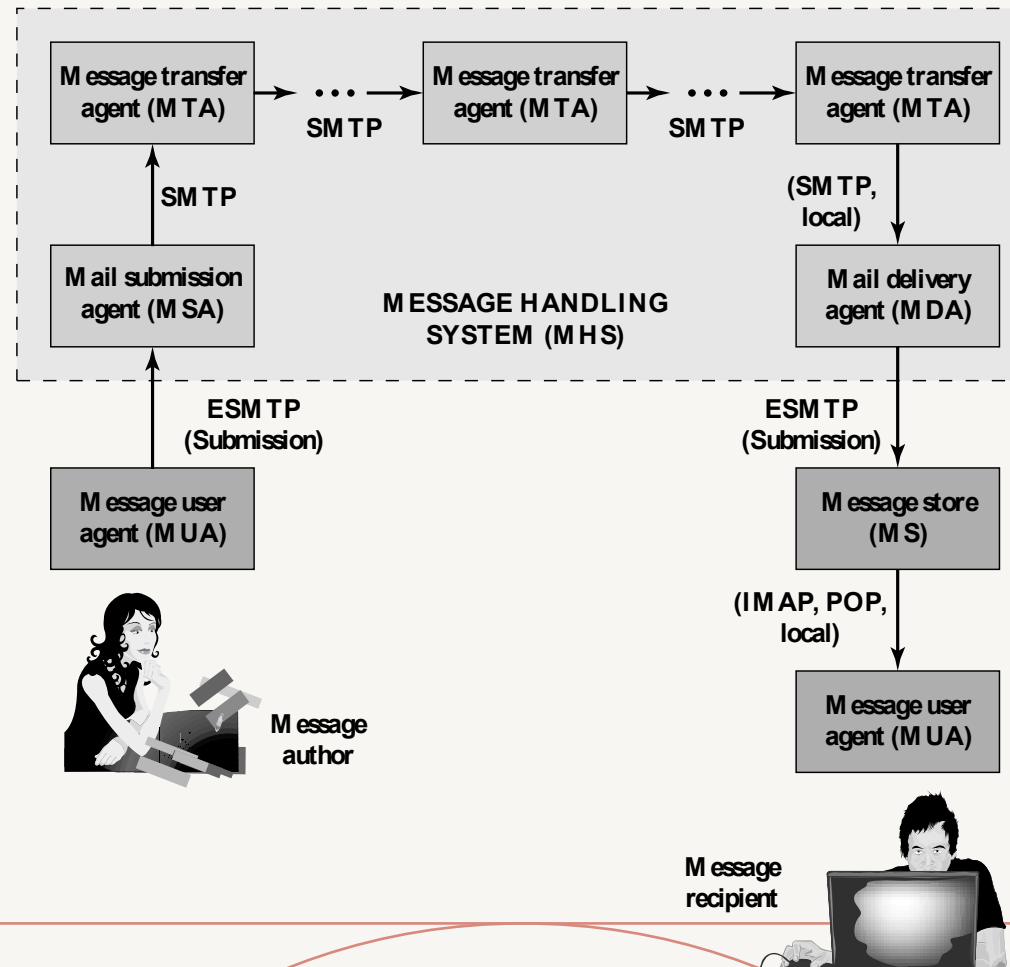
Passkeys

- Passkeys are 'syncable' WebAuthn credentials
- The private key is encrypted and backed up to the cloud (iCloud, Google, MS)
- Enables cross-device availability (User logs in on phone, then laptop)
- End-to-End Encryption ensures cloud providers cannot read the key
- Relationship: Passkeys are a specific, user-friendly implementation of WebAuthn:
 - WebAuthn is the Protocol (The 'API')
 - Passkey is the Implementation (The 'Product')
- Windows: Uses TPM 2.0. Private key generated inside hardware secure module.
- Apple (FaceID/TouchID): Uses Secure Enclave. Biometric unlock releases the signature operation

Passkeys

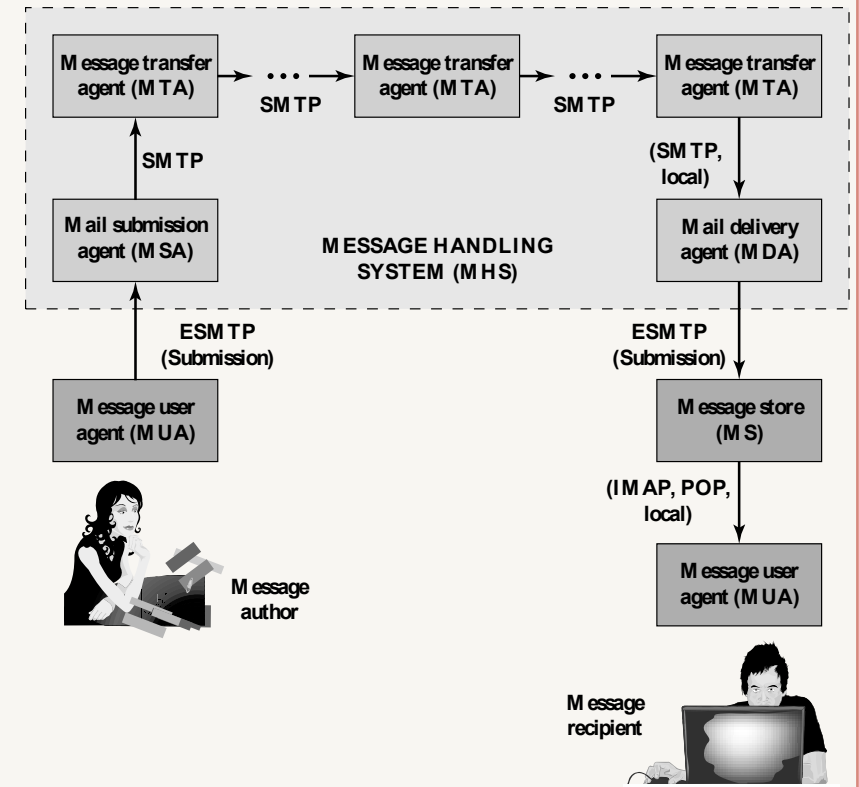
- Let's try a demo: webauthn.io
- To understand more, you can read this paper:
<https://dl.acm.org/doi/pdf/10.1145/3778360>

How Email Works



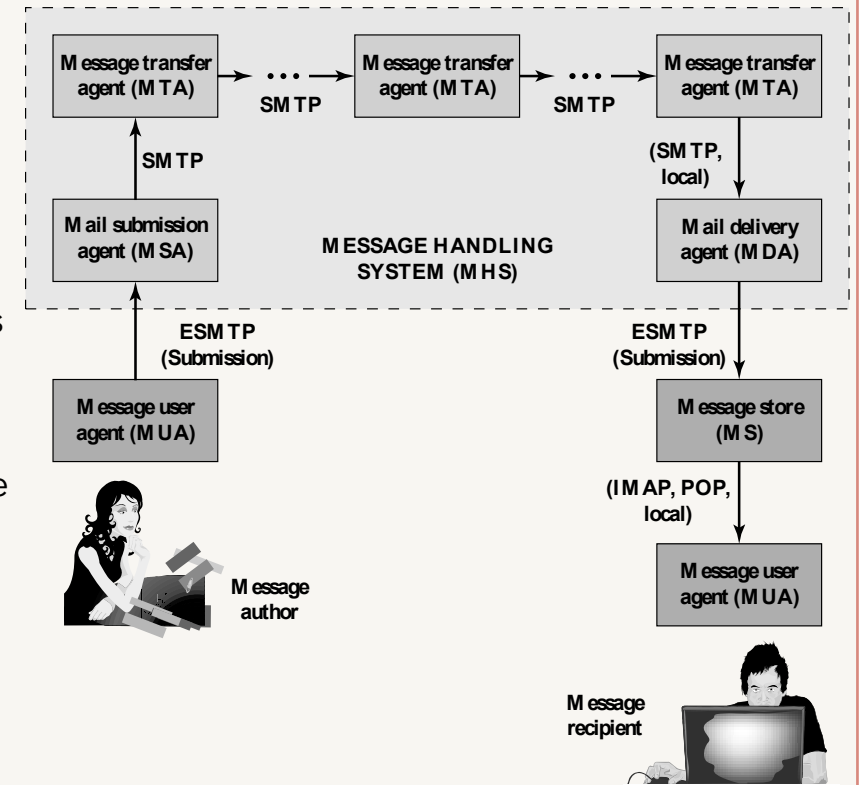
How Email Works

- Message User Agent (MUA):
 - Operates on behalf of user actors
 - It is the application by which a user composes and sends/receives an email
 - E.g. Outlook, Thunderbird
- Message Handling System (MHS):
 - An entity which is responsible for handling emails
 - Has a number of subcomponents: Mail Submission Agent (MSA), Message Transfer Agent (MTA) & Mail Delivery Agent (MDA)



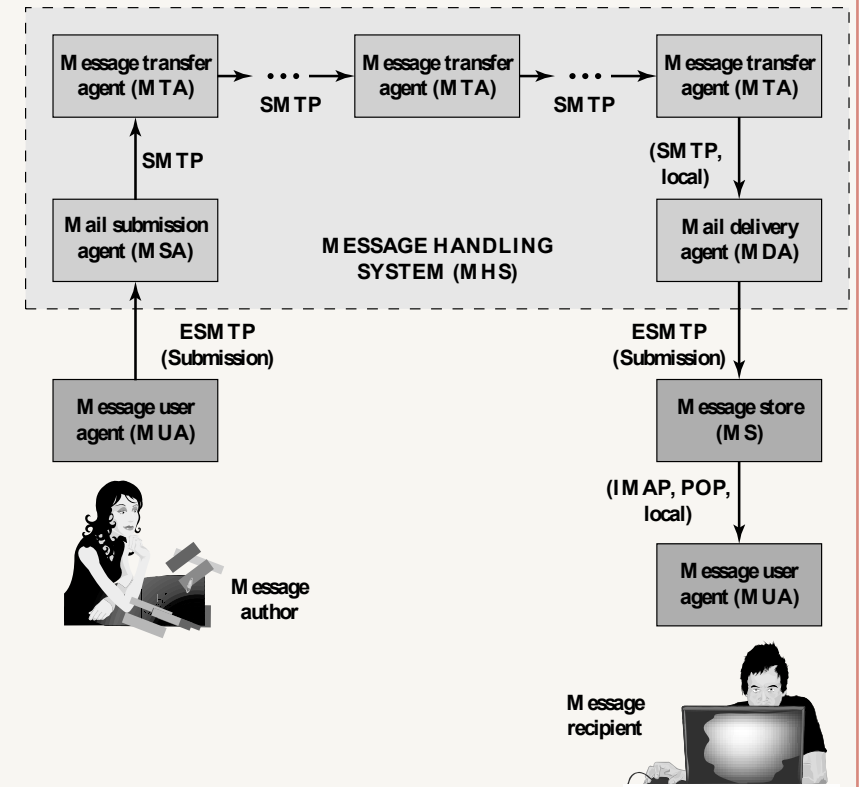
How Email Works

- Mail Submission Agent (MSA):
 - Accepts the message submitted by an MUA
 - Enforces the policies of the hosting domain
 - Forwards the message to an MTA
- Message Transfer Agent (MTA):
 - The MTA relays mail for one application-level hop
 - Relaying is performed by a sequence of MTAs, until the message reaches a destination MDA
- Mail Delivery Agent (MDA):
 - The MDA is responsible for transferring the message from the MHS to the MS after enforcing policies
- Message Store (MS):
 - An MS can be located on a remote server or on the same machine as the MUA
 - Receives an email from the MDA and delivers it to the receiver's MUA



How Email Works

- Two types of protocols are used for transferring emails
- The first type is used to move messages through the Internet from source (MSA) to destination (MDA)
 - The protocol used for this purpose is SMTP (Simple Mail Transfer Protocol)
- The second type consists of protocols used to transfer messages from the MS to the MUA
 - IMAP (Internet Mail Access Protocol (IMAP) and POP (Post Office Protocol) are the most commonly used for this purpose



How Email Works: SMTP

- SMTP is a text-based client-server protocol where
 - the client (email sender) contacts the server (next-hop recipient) and
 - issues a set of commands to tell the server about the message to be sent, then sending the message itself.
- The operation of SMTP consists of a series of commands and responses exchanged between the SMTP sender and receiver
- The majority of these commands are ASCII text messages sent by the client and a resulting return code (and additional ASCII text) returned by the server

How Email Works: SMTP

- The operation starts with the SMTP sender, who establishes the TCP connection on port 25
- Once the connection is established, the SMTP sender sends commands over the connection to the receiver
- Each command consists of a single line of text, beginning with a four-letter command code followed in some cases by an argument field
- Each command generates exactly one reply from the SMTP receiver
- Most replies are a single-line, although multiple-line replies are possible
- Each reply begins with a three-digit code and may be followed by additional information

How Email Works: SMTP

Once the TCP connection is established, Receiver's domain

```
S: 220 foo.com Simple Mail Transfer Service Ready
C: HELO bar.com
S: 250 OK
C: MAIL FROM:<Smith@bar.com>
S: 250 OK
C: RCPT TO:<Jones@foo.com>
S: 250 OK
C: RCPT TO:<Green@foo.com>
S: 550 No such user here
C: RCPT TO:<Brown@foo.com>
S: 250 OK
C: DATA
S: 354 Start mail input; end with <crLf>.<crLf>
C: Blah blah blah . . .
C: . . . etc. etc. etc.
C: <crLf><crLf>
S: 250 OK
C: QUIT
S: 221 foo.com Service closing transmission channel
```

HELO message identifies the sender's domain

TCP connection is terminated

How Email Works: POP3/IMAP

- Post Office Protocol (POP3) allows an email client (user agent) to download an email from an email server (MS)
- POP3 user agents connect via TCP to the server (typically port 110)
- The user agent enters a username and password (either stored internally for convenience or entered each time by the user for stronger security)
- After authentication, the UA can issue POP3 commands to retrieve and delete mail
- As with POP3, Internet Mail Access Protocol (IMAP) also enables an email client to access mail on an email server
- IMAP also uses TCP, with server TCP port 143
- IMAP is more complex than POP3 as it provides stronger authentication than POP3 and provides other functions not supported by POP3

ANY QUESTIONS?

